



# Governed & Scalable AI-assisted Analytics with dbt

Building AI acceleration you can actually trust



Jessica Stayton, Raini Laughlin

**Use private/incognito session**

**Workshop Registration page:**

[tinyurl.com/dbt-workshop-setup](https://tinyurl.com/dbt-workshop-setup)

- Select **Governed and scalable AI-assisted analytics with dbt**
- Enter the Passcode: **Summit2026!**
- **Take note:** User ID and Password

Connect to the conference wi-fi:

Network: dbt Summit

Passcode: run\_dbt!

Once you're seated, please join the WiFi – this  
helps everyone connect smoothly!

**Access AI closed captions and  
translation on your personal device.**



# Meet the speakers



**Jessica Stayton**  
(she/her)

Senior Resident  
Solutions Architect



**Raini Laughlin**  
(they/them)

Senior Technical  
Instructor

# Meet the TAs



**Carol Ohms**  
(she/her)

Senior Resident Solutions  
Architect



**Stephen Thibeault**  
(he/him)

Senior Resident Solutions  
Architect

## Meet the TAs



**Faith McKenna**  
(she/her)

Manager,  
Technical Instruction



**Saptarishi Baruah**  
(he/him)

Senior Solution Architect,  
Enterprise

# Learning Objectives

---

- Describe where AI helps most in analytics engineering, and where it introduces risk
- Apply a review framework for AI-assisted changes (correctness, performance, maintainability, documentation)
- Identify governance decisions needed before scaling AI usage (who can access what context, what outputs are acceptable, how changes get approved)
- Establish team norms that prevent "AI drift" in definitions, naming, and documentation

# What you'll leave with

---

## You'll be able to:

- Spot where AI helps vs. adds risk
- Set guardrails in dbt
- Make governance calls before you scale
- Prevent AI drift across a team

## Scope & where to go deeper:

- We govern within dbt + Wizard
  - not the whole industry

## Deeper elsewhere:

- *Creating context with dbt MCP Server (Thu)*
- *Accelerating analytics with AI (Thu)*
- Expert Lounge

# How we'll work together

## 100 All In!

- Stay engaged
- Participate



## Community-Minded!

- Dive in
- Engage in [dbt Community Slack](#)



## Be Curious!

- Ask questions
  - It's ok to be wrong
- Break stuff!

## You're encouraged to:

- Take care of your needs!
- Ask a neighbor

# Agenda

---

01

## Meet the Project

- sandbox account
- patterns
- break it

02

## AI Foundations

- meet Wizard

03

## Adding Governance to Development

- AGENTS.md
- building skills

04

## Governance Beyond Deployment

- CI gates and rubric
- failed job review

05

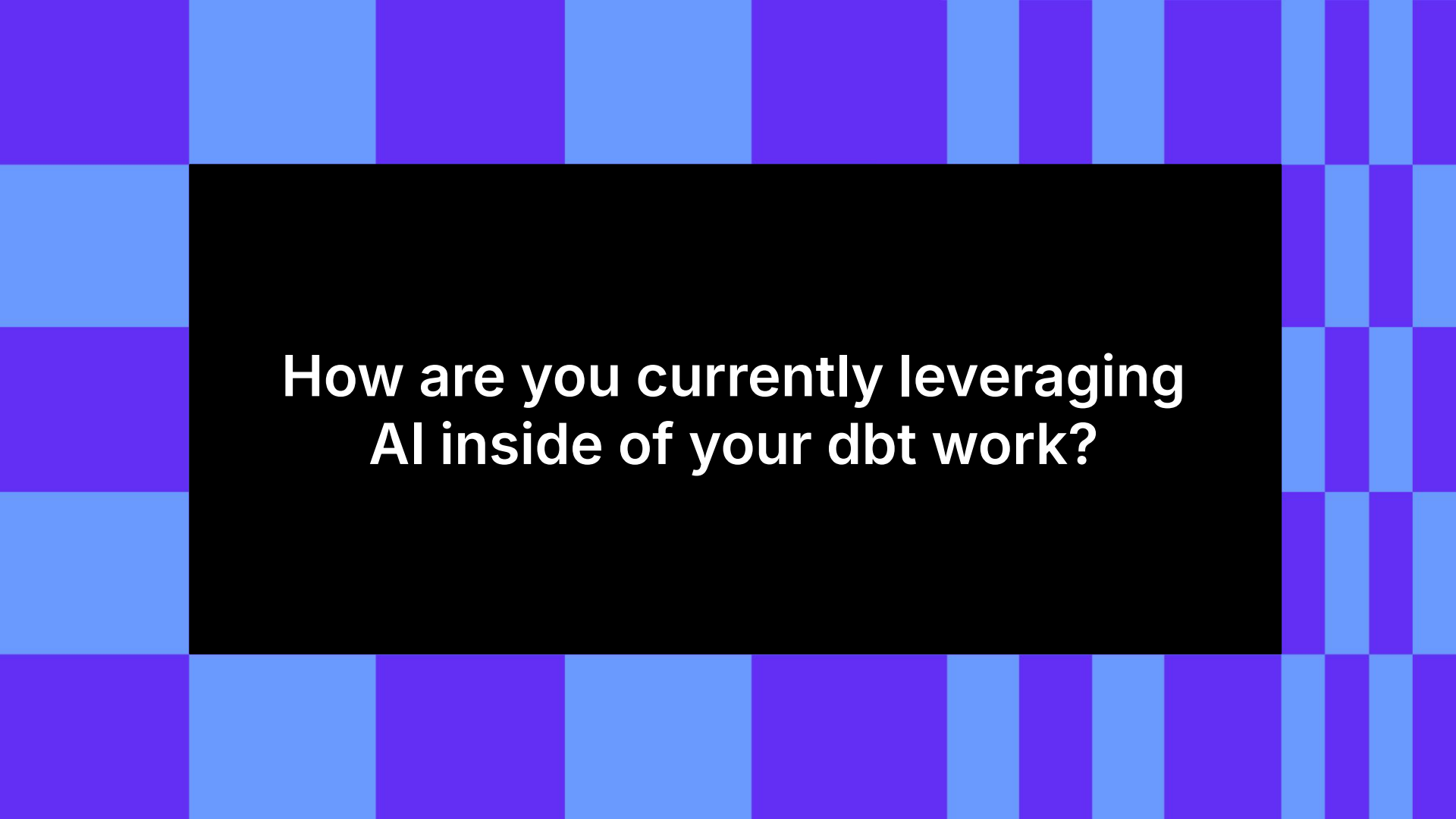
## Scaling

- between projects
- beyond dbt

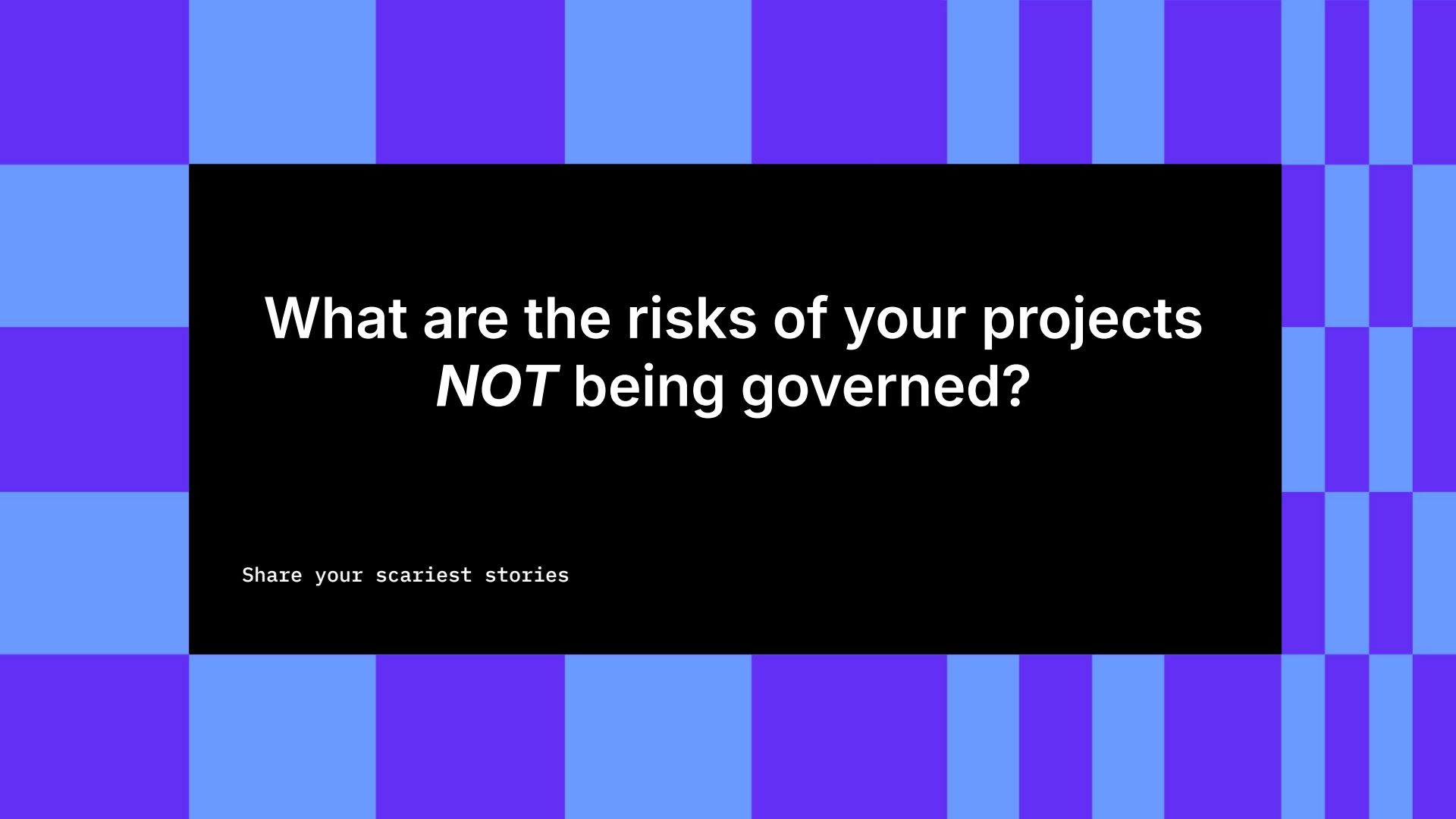
06

## Wrap-Up and Survey

- training survey



**How are you currently leveraging  
AI inside of your dbt work?**



# What are the risks of your projects *NOT* being governed?

Share your scariest stories



# What happens if you remove the guardrails?

Share your scariest stories





# Meet the Project...

...then break it!

# Sandbox Setup

**Log out** of any dbt accounts

- or use private/incognito session

**Workshop Registration page:**

[tinyurl.com/dbt-workshop-setup](https://tinyurl.com/dbt-workshop-setup)

- Select **Governed and scalable AI-assisted analytics with dbt**
- Enter the Passcode: **Summit2026!**
  - **Take note:** User ID and Password
    - Take a picture or screenshot 
- Create a new branch **first\_last\_summit26**



## Workshop Signup

Please provide your first name, last name, and which workshop you will be attending. After submitting this form, you may log into the new account for your workshop.

First Name

Last Name

Workshop

Passcode

☐ I agree to dbt's [Terms of Service](#)

 Complete Registration

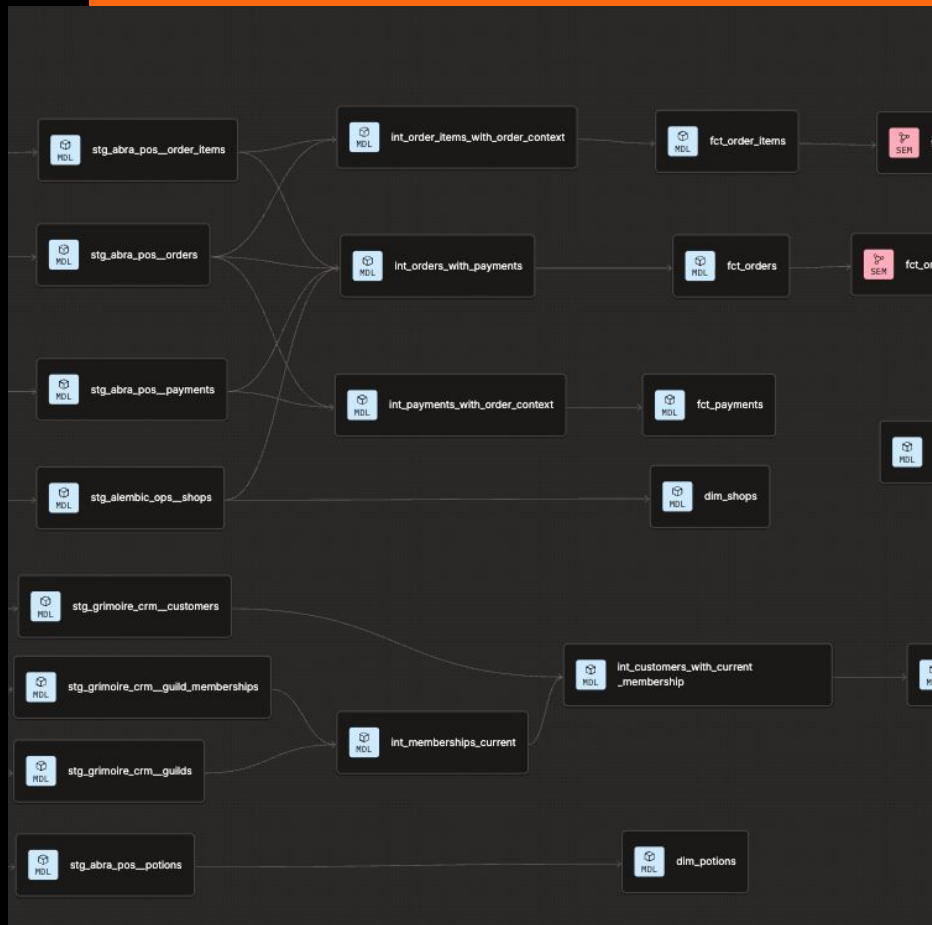
# Meet the project

## Welcome to Merlin & Co. Apothecaries!

The famous wizard, *Merlin*, has opened a shop of potions and other apothecary items!

Your job is to complete the pipeline:

- Built end-to-end: **abra** & **grimoire**
- Missing: **alembic** (alembic\_ops source)

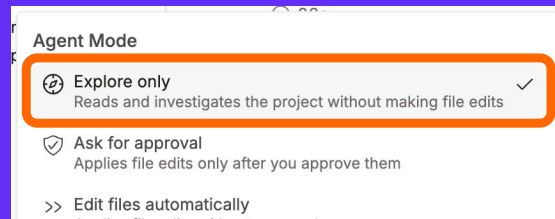


# Onboard to the Project

Hands-on  
5 Minutes



**Explore** → Plan → Implement → Verify



## Explore:

- On your own or prompt Wizard
  - docs/prompts/
  - README
  - docs/merlinco/
  - models/

## Hunt the patterns:

- naming, folders, tests, docs, grain, project context, etc.

## Write patterns in a scratchpad



# Our Project

Tell me about it...

```
models/
├─ staging/           # completed source-facing models and source declarations
├─ intermediate/     # completed join, fanout, and aggregation patterns
├─ marts/            # completed contracted marts and semantic definitions
├─ warlock/          # trainee baseline: staging/intermediate/marts
├─ wizard/           # trainee governed build: staging/intermediate/marts
└─ answer_key/       # disabled facilitator comparison models
macros/              # shared cleaning macros (to_boolean, copper_to_gold, conform_region, ...)
seeds/medium_data/   # portable raw CSV fixtures for facilitator/environment setup
docs/merlinco/
├─ ERD.md            # full schema diagram (columns, types, PK/FK markers)
├─ DATA_DICTIONARY.md # per-table column notes and deliberate data quirks
└─ STYLE_GUIDE.md    # modeling + naming conventions
```

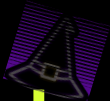
# Meet dbt Wizard

**Grounded in your dbt project through a native metadata engine**

**An AI agent harness built for governed analytics engineering**

- **dbt platform, the terminal, & Wizard Desktop App**
- understands lineage, tests and contracts, run results, and semantic definitions
- investigate → change → validate → review
- validates in your development schema
- shows a visual DAG of changes
- keeps governance and audit trails on by default





# Wizard navigates, repo defines

**Native context helps Wizard understand the project**

**Wizard has strong dbt-native grounding**

**AGENTS.md, routing, skills, workflows, and review rules encode decisions that should persist**

| Wizard brings                | Your team must define    |
|------------------------------|--------------------------|
| Project metadata             | Naming conventions       |
| Lineage and impact awareness | Grain and business rules |
| Tests and contracts          | When to stop and ask     |
| Validation workflow          | Approval and ownership   |
| Runtime context              | Reusable team workflows  |

# Now Break It!

Hands-on  
4 Minutes



Explore → Plan → **Implement** → Verify

## Prompt Wizard:

Use the prompt we have supplied for you!

>> `docs/prompts/warlock_build_prompt.md`

- Watch what it does
- Don't fix it, just observe
- Inspect what was built
  - `dbt ls -select tag:warlock`

### Agent Mode

- ☐ Explore only  
Reads and investigates the project without making file edits
- ☐ Ask for approval  
Applies file edits only after you approve them
- ☒ Edit files automatically  
Applies file edits without approval

# What happened?

Hands-on  
2 Minutes



## Compare with your neighbor:

- Turn to your neighbor
  - did you get the same output?

## Name the risks:

- What could native business and source context help with?
- Where did your outputs diverge?
- Which decisions should become shared policy?



# We need Governance!

---

## Three layers:

### Guide

- development files (AGENTS.md, skills)

### Enforce

- Human-in-the-loop
- Review before promotion
- CI/CD gates on every PR

### Runtime

- deployment, semantic layer, mesh

## Two ways AI meets dbt:

- AI that helps builds your dbt project
- dbt as the trusted context AI consumes



# AI Foundations

Name what we just felt

# The 80/20 framing

---

## High Volume (80%)

*Safe to Delegate to AI Tools*

- Scaffolding and boilerplate
- First pass docs and test stubs
- First draft of semantic metrics
- Syntax
  - window functions
  - regex
  - date spines

## High Stakes (20%)

*Human in the Loop*

- Business logic and edge cases
- Grain and join semantics
- Naming, layer, what ships
- Performance tradeoffs (i.e. materialization)
- **Governance**

# To keep the 20%, Own it!

---

## Define what matters

- **Design:** source → trusted table or metric
- **Success criteria:** what "correct" means
- **Rules:**
  - business logic
  - grain
  - governance

## Choose the autonomy

- Ask by default
- Prompt-back when context is missing
- Sign-off before production
- Earn auto for low-stakes, tested, reversible work

## Verify the result

- Prefer evidence over plausibility
- Keep changes small and reviewable
- Successful job ≠ numbers are right
- **You** own what ships

# Why AI misbehaves

---

## AI Drift

- Same prompt, different answers each run
  - You saw this with your neighbor

## Contributions

- Personal memory files
- User habits
- Software updates

## AI Hallucinations

- Single instance of false confidence
  - e.g. a column that doesn't exist

## Contributions:

- Gaps in training
- Guesswork instead of fact retrieval

# Work the pattern

---

*Treat AI as a collaborator, not a vending machine*

## Explore:

- Map the source
- Flag the casts/renames
- List assumptions *before* any SQL

## Implement:

- Build in small and reviewable chunks

## Plan:

- Propose:
  - Columns
  - Grain
  - Schema.yml
- Human checkpoint for "done"

## Verify:

- compile/build
- Check grain and row counts
- Add a test that AI couldn't write



# Adding Governance to Development

Institutionalizing knowledge to prevent drift

# Spoiler Alert, you started this

---

## Remember the patterns you jotted down in Section 2?

- Notes are in your scratch pad... let's clean them up!
- Formalize:
  - layers
  - naming conventions
  - grain rules
  - where to look
- Your notes were governance all along!

# Proof of Life

---

## You have to tell AI when to check in!

- When you feed it an under-documented source...
  - what happens?
- Good skill:
  - stops and asks instead of guessing
- Add prompt-backs:
  - "if context is missing, ask before building"

# Build the AGENTS.md

Hands-on  
5 Minutes

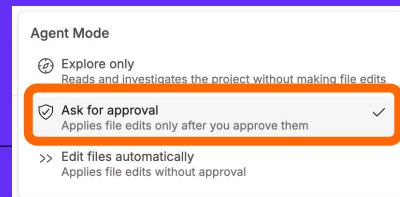


 Explore → Plan → Implement → Verify

## Format the AGENTS.md

- document discovered patterns
- follow the TODOs
- Include:
  - layers
  - naming
  - grain
  - documentation & testing rules
  - prompt-back requests

## Use dbt Wizard to help format



# The files that guide AI

---

## AGENTS.md

- what the agent should know/do
  - context, layers, naming, grain

## SECURITY.md

- what the agent must **not** do:
  - no prod writes, no creds, stop-and-ask

## skills/

- reusable, load-when-relevant instructions

## ROUTING.md

- maps how your skills get used together

## How to leverage them

- One shared source of truth  
→ less drift
- Wizard reads AGENTS.md  
+ skills by default

# Project inheritance

Hands-on  
3 Minutes



In `.agents/`

- Review all skills/
- Review ROUTING.md

 **Ask Wizard or yourself:**

- What workflows are already in place?
- To reliably build out `alembic_ops`, what do *you* think we'll need?



Explore → Plan → Implement → Verify

# Build a skill from a pattern

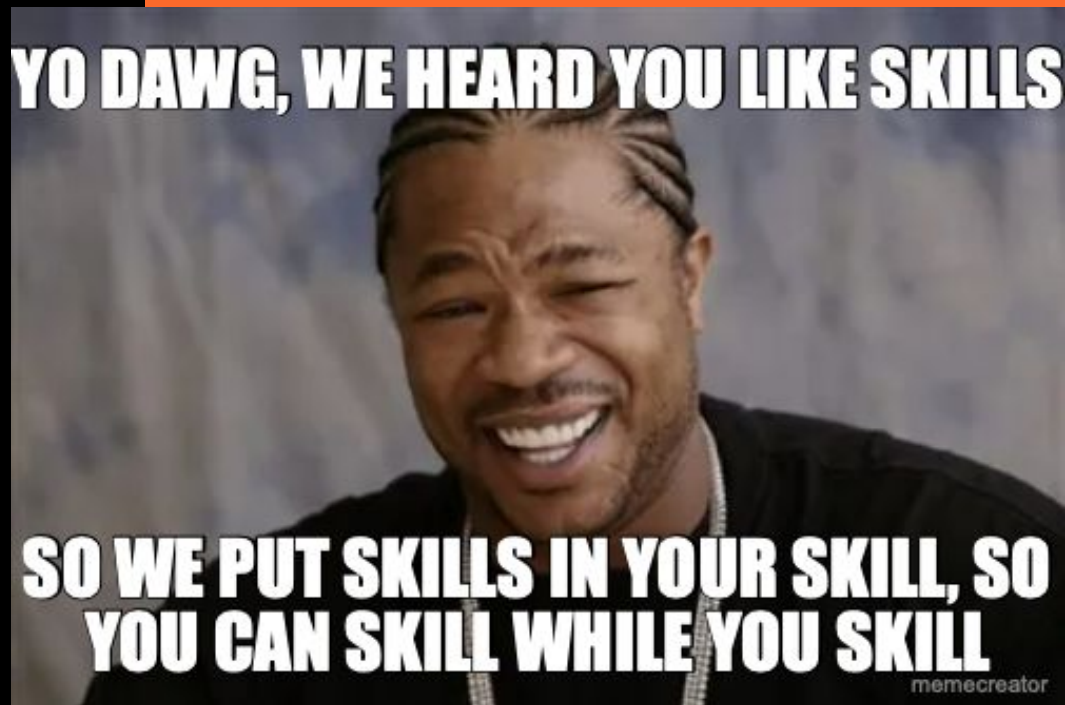
## A skill for writing skills

Point it at the staging pattern you already found

### Output:

- a reusable "build a staging model" skill

*Capture the pattern once → anyone reproduces it*



# Plan the Alembic path

---

**Staging:** What is the source grain?

**Intermediate:** What transformations are reusable?

**Mart:** What decision or metric does this serve?

**Plan first. Implement in small, reviewable steps.**

# Skill Building

Use the building-governed-skills skill to build a skill!

Prompt Wizard to build a skill for the staging layer

- Given a raw source, build a staging layer this way:
  - file name convention
  - CTEs
  - source() macro
  - renames
  - casts
  - tests
  - contracts
  - normalize nulls/casing/values
- Test it out

## Agent Mode

-  Explore only  
Reads and investigates the project without making file edits
-  Ask for approval  
Applies file edits only after you approve them
-  Edit files automatically  
Applies file edits without approval

```
1 # Staging model checklist
2
3 ## Source grounding
4
5 - [ ] Source declaration, raw-table grain, key, and important FKs were inspected.
6 - [ ] Actual source values were profiled when type, null, casing, timestamp, boolean, money, or catego
7 - [ ] The source model name follows `stg_<source>__<entity>`.
8 - [ ] The model reads exactly one `source()`.
9
10 ## Layer boundary
11
12 - [ ] The model preserves one row per raw source row.
13 - [ ] No joins, cross-record deduplication, aggregation, reporting logic, or metric calculation appear
14 - [ ] Renames, casts, format cleanup, and conformance belong at this layer.
15 - [ ] Shared macros are reused for known cleanup patterns.
16
17 ## Data product hygiene
18
19 - [ ] Key fields, booleans, dates/timestamps, and money fields use project naming and types.
20 - [ ] Raw money integers are retained and the reporting-currency field is exposed when applicable.
21 - [ ] Normalized categorical fields have grounded accepted values where appropriate.
22 - [ ] Source and staging descriptions explain grain/meaning rather than restating names.
23
24 ## Evidence
25
26 - [ ] Source and staging tests cover the key and material data-quality rules.
27 - [ ] Output was checked for row count, key uniqueness, nulls, casts, and category normalization.
28 - [ ] Scoped build and SQLFluff results are recorded.
29 - [ ] Any unresolved business mapping, unit conversion, null policy, or source-authority question was |
30
```

# Build Layering Skills

Hands-on  
10 Minutes



## Plan & Build Skills for Each Layer

Use the prompt template provided in docs/prompts.

- **Staging**
  - *We just built this together!*
- **Intermediate**
  - What transformations are reusable?
- **Mart**
  - What decision or metric does this serve?

## Write Skills for Intermediate & Marts

- Build the skill for each layer
- Validate your new skills:
  - Does the skill exist?
  - Is its route and owner are explicit;
  - Does it stop and prompt back when clarification is needed?
  - Does it require a scoped build plus warehouse checks to verify accuracy?



# You built the guardrails, now watch them work

---

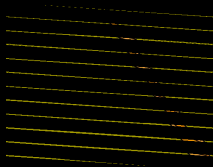
**You built** intermediate + marts **skills!**

**Our planning skill** can now generate the build plan for us to approve

**Our orchestrator** can use these skills to build the alembic source

**This has been the goal all along!**

- Craft the build plan
- watch your guardrails work!



# Now Fix It!

Hands-on  
10 Minutes



Explore → Plan → **Implement** → Verify

## Prompt Wizard

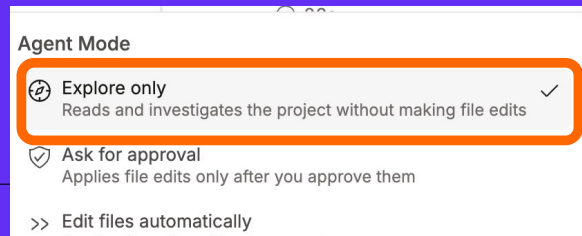
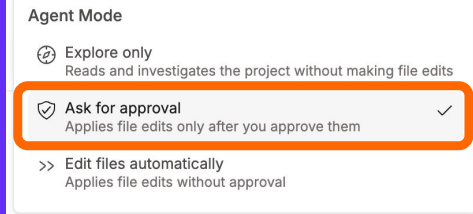
Run the prompt in

`docs/prompts/run_alembic_[plan/build]  
_prompt.md`

Or use ours to guide you to write your own prompt!

## Compare Wizard and Warlock outputs:

- naming
- tests
- documentation
- grain
- contracts



# Same build, now governed

## More refined instructions

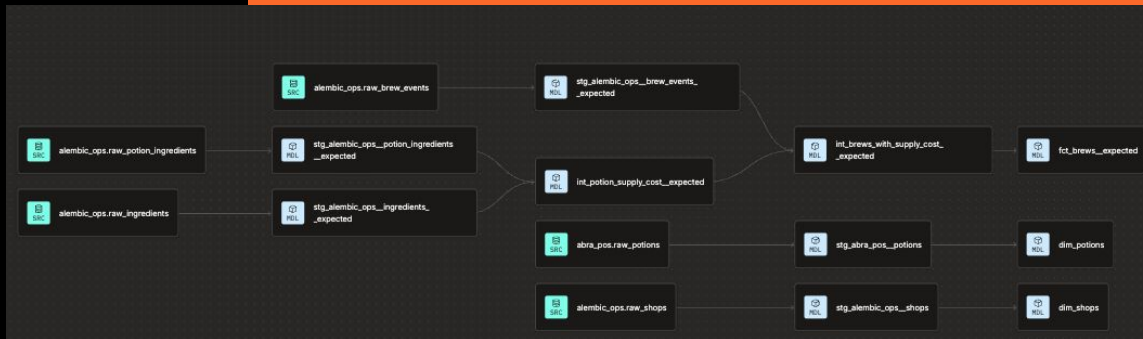
- now with guardrails in place

## Convention-correct

- naming, tests, documented grain

## Fewer stops

- fewer corrections, output you can trust



This is the difference **governance** makes

# How to review

---

## Validation for how we know it worked:

- ✓ Files parse and compile
- ✓ Models have the expected grain
- ✓ Data quality tests pass
- ✓ Documentation and contracts are present
- ✓ The diff is small and reviewable
- ✓ Human approval and independent controls remain visible

# Govern the rules like code

---

## The question:

*Who can edit the rules vs.  
propose a change, and on what?*

### Shared assets

- AGENTS.md
- SECURITY.md
- skills

## The answer:

*Review config the same way  
you review code*

- CODEOWNERS → owner notified on every change
- Guardrails are only trustworthy if they're governed too!



# Governance beyond Development

Explicit boundaries for risk and action

# Guardrails in action

---

**Building with AI is one thing, shipping what it built is another**

- Now:
  - review
  - promotion
  - production
- The stakes rise the closer you get to prod
- Same discipline, higher-consequence surface

# CI gates & agentic PR review

CI checks business logic, not just formatting

- An agent can review the PR
  - a human still signs off
- Wizard auto-tags its sessions  
→ audit trail for free

Let me work



My magic

# Debug a failed job with Wizard

## A deploy job failed, now what?

- Click Debug with Wizard → diagnosis
- You decide the fix
  - *the agent doesn't act alone*
- Compare with your neighbor
  - same diagnosis?



Run #505475986



Debug with Wizard



This run failed. Help me debug it.

This run failed. Help me debug it.

Run ID: 505475986

Job ID: 1020133

Please pull the run details and the failure output, identify the root cause, and tell me what to try next.

> Reasoning

I'm pulling the job configuration, run steps, and focused failure output, then I'll verify the job belongs to the current project before tracing the root cause.

> Loaded dbt skill, got job details, and used 2 other tools



# Scaling: Consistency is an Infrastructure Problem

One person's pattern becomes shared infrastructure

# Consistency across teams

---

## You built a skill that works... for YOU

- Now: how does it work for the whole team?
- The problem isn't skill, it's consistency
- 40 developers, 40 private rule sets = drift at scale

# Scale within a project

---

## Make it Shared

### *shared assets:*

- Skills + AGENTS.md live in the repo
  - everyone gets them
- One shared PR-review agent
  - same rubric for all

## Make it Enforced

### *keep it owned:*

- CODEOWNERS notifies owners when rules change
- Propose via PR, not silent edits
- Same discipline as feature branches

# Across projects & beyond dbt

---

## Coming Soon

### *many projects / Mesh:*

- Sharing skills across projects  
→ native support coming
- Watch for announcements
  - don't build workarounds yet

## Beyond dbt

### *outside dbt:*

- MCP, Runlayer, plugins = the next frontier
  - see Creating context with dbt MCP Server session on Thurs
- NEW! **Agents Schema\*\*\***
- On-demand course coming this fall

# Scale the pattern, not the chaos

---

## Monday morning: start small

### Three steps:

- **Commit the baseline**
  - AGENTS.md, SECURITY.md, routing, dbt Wizard
- **Govern one workflow**
  - explore → plan → implement → verify
- **Measure and improve**
  - Use the governance scorecard to find the next gap



# Wrap-up and Next Steps

We did it!

# Recap the arc

---

- **Explore** → felt the risk
- **Named it:** 80/20, drift, the AI workflow
- **Fixed the inputs:** AGENTS.md + skills
- **Governed the outputs:** review, sign-off, audit trail
- **Scaled the pattern,** not the chaos

*LLMs reduce typing, not responsibility*

Thank you!



Training Survey

[dbtlearn.typeform.com/dbt-summit-labs](https://dbtlearn.typeform.com/dbt-summit-labs)

 SUMMIT